

```
In [1]: # Behzad Ghabaei
        # CS 82A - Data Science
        # module3_lab2.ipynb
        # Textbook Problem 2.1
        # Outliers in the crime data
        # Instructor Seno
        # 9/16/2026
```

The dataset is called "usarrests.csv" and has statistics for arrests per 100,000 residents for assault and murder, among the 50 states in 1973. The percentage of the population living in urban areas is given. UrbanPop is a percentage out of 100. None should exceed 100%.

```
In [2]: import pandas as pd
        df = pd.read_csv("usarrests.csv")
```

Step 1: Load the dataset and size up the file

The four commands you should always run first are: **df.head()** **df.shape** **df.info()** **df.describe()**. Since we have a small dataset we can use head() to see all of our data.

```
In [3]: df.head(50)
```

Out[3]:

	Unnamed: 0	Murder	Assault	UrbanPop
0	Alabama	13.2	236.0	58
1	Alaska	10.0	263.0	48
2	Arizona	8.1	294.0	80
3	Arkansas	8.8	190.0	50
4	California	9.0	276.0	91
5	Colorado	7.9	204.0	78
6	Connecticut	3.3	110.0	77
7	Delaware	5.9	238.0	72
8	Florida	15.4	335.0	80
9	Georgia	17.4	NaN	60
10	Hawaii	5.3	46.0	83
11	Idaho	2.6	120.0	54
12	Illinois	10.4	249.0	83
13	Indiana	7.2	113.0	65
14	Iowa	2.2	56.0	570
15	Kansas	6.0	115.0	66
16	Kentucky	9.7	109.0	52
17	Louisiana	15.4	249.0	66
18	Maine	2.1	83.0	51
19	Maryland	11.3	300.0	67
20	Massachusetts	4.4	149.0	85
21	Michigan	12.1	255.0	74
22	Minnesota	2.7	72.0	66
23	Mississippi	16.1	259.0	44
24	Missouri	9.0	178.0	70
25	Montana	6.0	109.0	53
26	Nebraska	4.3	102.0	62
27	Nevada	12.2	252.0	81
28	New Hampshire	2.1	57.0	56
29	New Jersey	7.4	159.0	89
30	New Mexico	11.4	285.0	70
31	New York	11.1	254.0	6
32	North Carolina	13.0	337.0	45
33	North Dakota	0.8	45.0	44
34	Ohio	7.3	120.0	75
35	Oklahoma	6.6	151.0	68

	Unnamed: 0	Murder	Assault	UrbanPop
36	Oregon	4.9	159.0	67
37	Pennsylvania	6.3	106.0	72
38	Rhode Island	3.4	174.0	87
39	South Carolina	14.4	879.0	48
40	South Dakota	3.8	86.0	45
41	Tennessee	13.2	188.0	59
42	Texas	12.7	201.0	80
43	Utah	3.2	120.0	80
44	Vermont	2.2	48.0	32
45	Virginia	8.5	156.0	63
46	Washington	4.0	145.0	73
47	West Virginia	5.7	81.0	39
48	Wisconsin	2.6	53.0	66
49	Wyoming	6.8	161.0	60

At first glance, we see our data is missing a value on line 9, Georgia's "Assault" category, we also see an UrbanPop percentage at 570 for Iowa (strange number). Next, we see North Dakota has .8 Murder which would round to 0, when we change the data types. The data types are floating numbers, which is too noisy. Int would work better, to save storage. New York's UrbanPop looks too low at 6%. This seems to be more of a 60% than 6% considering New York's reputation of a dense population.

The shape() confirms the size. 50 rows , 4 columns

```
In [4]: df.shape
```

```
Out[4]: (50, 4)
```

The data types and missing values are caught here. Assault shows us 49/50

```
In [5]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Unnamed: 0   50 non-null     str
1   Murder       50 non-null     float64
2   Assault      49 non-null     float64
3   UrbanPop     50 non-null     int64
dtypes: float64(2), int64(1), str(1)
memory usage: 1.7 KB
```

It is useful to know the max, min, mean and median, for filling missing values.

```
In [6]: df.describe()
```

```
Out[6]:
```

	Murder	Assault	UrbanPop
count	50.00000	49.000000	50.00000
mean	7.78800	182.183673	74.20000
std	4.35551	130.877435	73.40828
min	0.80000	45.000000	6.00000
25%	4.07500	109.000000	53.25000
50%	7.25000	159.000000	66.00000
75%	11.25000	249.000000	77.75000
max	17.40000	879.000000	570.00000

From describe() several outliers are visible such as UrbanPop min at 6% and max at 570%. This is not possible.

What are the data types we're working with. How can we reduce the noisy data. Murder and Assault are float data types.

```
In [7]: df.dtypes
```

```
Out[7]: Unnamed: 0      str
Murder      float64
Assault      float64
UrbanPop     int64
dtype: object
```

We know the correct spelling of our columns

```
In [8]: df.columns
```

```
Out[8]: Index(['Unnamed: 0', 'Murder', 'Assault', 'UrbanPop'], dtype='str')
```

Step 2: Write your first observations

Looking at the data, many concerns arise about the precision of the data. First, using floats may not be a good idea, because a death is either true or false. Using integers will reflect the true nature of deaths. Integers will eliminate noise and save storage. Urban populations should not exceed 100% and if they are integers, it works best for clarity, instead of decimal point values. An assault could be used as an integer, which would be best, so that decimal assaults don't cause confusion. Lets change the data types of each of these to integers.

Next, lets keep the urban population percentage limited and cut off at 100. Looking at impossible and extreme values of each entry is done with df.head(50) which can list them all.

Assault is missing a value, that should be filled.

Step 3: Count the missing values

```
In [9]: # apply tutorial 1 to the whole dataframe
df.isna().sum() # 1 missing value under assault
```

```
Out[9]: Unnamed: 0      0
        Murder      0
        Assault      1
        UrbanPop      0
        dtype: int64
```

Step 4: Impute the missing value, dropping rows loses valuable data.

```
In [10]: df.shape # run this before and after filling missing values.
```

```
Out[10]: (50, 4)
```

```
In [11]: # Fill the missing prices using the Tutorial 2 pattern
# Looking for: Impossible Values, Suspicious Outliers, Missing Values, Floating Point Noise
fill_value = df["Assault"].median() # The 50% median of "Assault" when running df.describe() is 159.
# whether mean or median fits your Lab data better
df["Assault"] = df["Assault"].fillna(fill_value)
print(df.head(20))
```

	Unnamed: 0	Murder	Assault	UrbanPop
0	Alabama	13.2	236.0	58
1	Alaska	10.0	263.0	48
2	Arizona	8.1	294.0	80
3	Arkansas	8.8	190.0	50
4	California	9.0	276.0	91
5	Colorado	7.9	204.0	78
6	Connecticut	3.3	110.0	77
7	Delaware	5.9	238.0	72
8	Florida	15.4	335.0	80
9	Georgia	17.4	159.0	60
10	Hawaii	5.3	46.0	83
11	Idaho	2.6	120.0	54
12	Illinois	10.4	249.0	83
13	Indiana	7.2	113.0	65
14	Iowa	2.2	56.0	570
15	Kansas	6.0	115.0	66
16	Kentucky	9.7	109.0	52
17	Louisiana	15.4	249.0	66
18	Maine	2.1	83.0	51
19	Maryland	11.3	300.0	67

```
In [12]: df.shape
```

```
Out[12]: (50, 4)
```

```
In [13]: df.isnull().sum()
```

```
Out[13]: Unnamed: 0      0
        Murder      0
        Assault      0
        UrbanPop      0
        dtype: int64
```

Assault no longer has a null value. The 50 percentile replaced the NaN.

Step 5: Remove duplicate rows

```
In [14]: print(df.duplicated().sum()) # counts fully repeated rows
df = df.drop_duplicates() # removes them.
```

0

No duplicates are detected.

Step 6: Repair the data types of each float

```
In [15]: df["Murder"] = df["Murder"].astype(int)
df["Assault"] = df["Assault"].astype(int)
# df["UrbanPop"] = df["UrbanPop"].astype(int) # already an int type
```

```
In [16]: df.dtypes
```

```
Out[16]: Unnamed: 0      str
Murder      int64
Assault     int64
UrbanPop    int64
dtype: object
```

```
In [17]: df.describe()
```

```
Out[17]:
```

	Murder	Assault	UrbanPop
count	50.000000	50.000000	50.000000
mean	7.420000	181.720000	74.200000
std	4.412829	129.576554	73.40828
min	0.000000	45.000000	6.000000
25%	4.000000	109.000000	53.25000
50%	7.000000	159.000000	66.00000
75%	11.000000	249.000000	77.75000
max	17.000000	879.000000	570.00000

```
In [18]: df.head(50)
```

Out[18]:

	Unnamed: 0	Murder	Assault	UrbanPop
0	Alabama	13	236	58
1	Alaska	10	263	48
2	Arizona	8	294	80
3	Arkansas	8	190	50
4	California	9	276	91
5	Colorado	7	204	78
6	Connecticut	3	110	77
7	Delaware	5	238	72
8	Florida	15	335	80
9	Georgia	17	159	60
10	Hawaii	5	46	83
11	Idaho	2	120	54
12	Illinois	10	249	83
13	Indiana	7	113	65
14	Iowa	2	56	570
15	Kansas	6	115	66
16	Kentucky	9	109	52
17	Louisiana	15	249	66
18	Maine	2	83	51
19	Maryland	11	300	67
20	Massachusetts	4	149	85
21	Michigan	12	255	74
22	Minnesota	2	72	66
23	Mississippi	16	259	44
24	Missouri	9	178	70
25	Montana	6	109	53
26	Nebraska	4	102	62
27	Nevada	12	252	81
28	New Hampshire	2	57	56
29	New Jersey	7	159	89
30	New Mexico	11	285	70
31	New York	11	254	6
32	North Carolina	13	337	45
33	North Dakota	0	45	44
34	Ohio	7	120	75
35	Oklahoma	6	151	68

	Unnamed: 0	Murder	Assault	UrbanPop
36	Oregon	4	159	67
37	Pennsylvania	6	106	72
38	Rhode Island	3	174	87
39	South Carolina	14	879	48
40	South Dakota	3	86	45
41	Tennessee	13	188	59
42	Texas	12	201	80
43	Utah	3	120	80
44	Vermont	2	48	32
45	Virginia	8	156	63
46	Washington	4	145	73
47	West Virginia	5	81	39
48	Wisconsin	2	53	66
49	Wyoming	6	161	60

Looking at the data, we have produced nice integer numbers, also, at a glance, it does not make sense that densely populated states have low populations but scarcely populated states have high ones, such as New York at 6, and Wyoming at 60. What can be done is replace a value. For example Iowa is 570 but might need to be at 57, and perhaps New York should be at 60, instead of 6. Lets change these 2 values. The Pandas.pydata.org "Cookbook" snippet helps us by showing how to replace values. (https://pandas.pydata.org/docs/user_guide/cookbook.html)

```
In [7]: df.loc[df.AAA < 5, ["BBB", "CCC"]] = 2000
```

```
In [8]: df
```

```
Out[8]:
```

	AAA	BBB	CCC
0	4	2000	2000
1	5	555	555
2	6	555	555
3	7	555	555

```
In [19]: # New York changed from 6 to 60.0 %
df.loc[31, "UrbanPop"] = 60
```

```
In [20]: # Iowa changed from 570 to 57 %
df.loc[14, "UrbanPop"] = 57
```

Also North Dakota has 0 murders which pulls down the average, however we know that South Dakota with a similar UrbanPop of 45 (North Dakota at 44) has 3 murders per 100,000. Lets change North Dakota's murder figure to the same as South Dakota.

```
In [21]: df.loc[33, "Murder"] = 3 # North Dakota is a negative outlier and probably skews the average.
```

These represent numbers of Murder or Assault per 100,000 residents in 1973.

```
In [22]: df.head(50)
```

Out[22]:

	Unnamed: 0	Murder	Assault	UrbanPop
0	Alabama	13	236	58
1	Alaska	10	263	48
2	Arizona	8	294	80
3	Arkansas	8	190	50
4	California	9	276	91
5	Colorado	7	204	78
6	Connecticut	3	110	77
7	Delaware	5	238	72
8	Florida	15	335	80
9	Georgia	17	159	60
10	Hawaii	5	46	83
11	Idaho	2	120	54
12	Illinois	10	249	83
13	Indiana	7	113	65
14	Iowa	2	56	57
15	Kansas	6	115	66
16	Kentucky	9	109	52
17	Louisiana	15	249	66
18	Maine	2	83	51
19	Maryland	11	300	67
20	Massachusetts	4	149	85
21	Michigan	12	255	74
22	Minnesota	2	72	66
23	Mississippi	16	259	44
24	Missouri	9	178	70
25	Montana	6	109	53
26	Nebraska	4	102	62
27	Nevada	12	252	81
28	New Hampshire	2	57	56
29	New Jersey	7	159	89
30	New Mexico	11	285	70
31	New York	11	254	60
32	North Carolina	13	337	45
33	North Dakota	3	45	44
34	Ohio	7	120	75
35	Oklahoma	6	151	68

	Unnamed: 0	Murder	Assault	UrbanPop
36	Oregon	4	159	67
37	Pennsylvania	6	106	72
38	Rhode Island	3	174	87
39	South Carolina	14	879	48
40	South Dakota	3	86	45
41	Tennessee	13	188	59
42	Texas	12	201	80
43	Utah	3	120	80
44	Vermont	2	48	32
45	Virginia	8	156	63
46	Washington	4	145	73
47	West Virginia	5	81	39
48	Wisconsin	2	53	66
49	Wyoming	6	161	60

Now lets take a look at describe for details. The count is the same, but the UrbanPop average has changed from 74 to 65. The min has changed from (New York 6) to (Vermont 32). The max has changed from (Iowa 570 to California 91). Has the median changed? No, it's still at 66.

```
In [23]: df.describe()
```

```
Out[23]:
```

	Murder	Assault	UrbanPop
count	50.000000	50.000000	50.000000
mean	7.480000	181.720000	65.020000
std	4.329491	129.576554	14.188944
min	2.000000	45.000000	32.000000
25%	4.000000	109.000000	54.500000
50%	7.000000	159.000000	66.000000
75%	11.000000	249.000000	76.500000
max	17.000000	879.000000	91.000000

```
In [24]: df[df["UrbanPop"] == 32] # this was the new min
```

```
Out[24]:
```

	Unnamed: 0	Murder	Assault	UrbanPop
44	Vermont	2	48	32

```
In [25]: df[df["UrbanPop"] == 91] # this was the new max
```

```
Out[25]:
```

	Unnamed: 0	Murder	Assault	UrbanPop
4	California	9	276	91

None of the UrbanPop percentages exceed 100 percent.

```
In [26]: df [ df["UrbanPop"] > 100 ]
```

```
Out[26]:
```

	Unnamed: 0	Murder	Assault	UrbanPop
--	------------	--------	---------	----------

We have successfully pre-processed the data. First addressing all the missing values. Second, looking for outliers and noisy data to smooth, with data type conversion and min, max, adjustments, and with percentage cut off at 100. Lastly, let's establish a relation between urban population category and crime type.

- Small < 50%
- medium < 60%
- large < 70%
- extra-large >= 70%

We will check [pandas.pydata.org](https://pandas.pydata.org/docs/user_guide/style.html) for suggestions on how to categorize datasets. "User Guide" > "Table Visualization" > here is the snippet of a function with the if-elif-else condition:

```
[2]:
def rain_condition(v):
    if v < 1.75:
        return "Dry"
    elif v < 2.75:
        return "Rain"
    return "Heavy Rain"

styler.format(rain_condition)
```

(https://pandas.pydata.org/docs/user_guide/style.html)

```
In [27]: def UrbanPop_condition(v):
    if v < 50:
        return "Small"
    elif v < 60:
        return "Medium"
    elif v < 70:
        return "Large"
    else:
        return "Extra-Large"
# the apply function was needed here
df['UrbanPop_Category'] = df["UrbanPop"].apply(UrbanPop_condition)
df.head(60)
```

Out[27]:

	Unnamed: 0	Murder	Assault	UrbanPop	UrbanPop_Category
0	Alabama	13	236	58	Medium
1	Alaska	10	263	48	Small
2	Arizona	8	294	80	Extra-Large
3	Arkansas	8	190	50	Medium
4	California	9	276	91	Extra-Large
5	Colorado	7	204	78	Extra-Large
6	Connecticut	3	110	77	Extra-Large
7	Delaware	5	238	72	Extra-Large
8	Florida	15	335	80	Extra-Large
9	Georgia	17	159	60	Large
10	Hawaii	5	46	83	Extra-Large
11	Idaho	2	120	54	Medium
12	Illinois	10	249	83	Extra-Large
13	Indiana	7	113	65	Large
14	Iowa	2	56	57	Medium
15	Kansas	6	115	66	Large
16	Kentucky	9	109	52	Medium
17	Louisiana	15	249	66	Large
18	Maine	2	83	51	Medium
19	Maryland	11	300	67	Large
20	Massachusetts	4	149	85	Extra-Large
21	Michigan	12	255	74	Extra-Large
22	Minnesota	2	72	66	Large
23	Mississippi	16	259	44	Small
24	Missouri	9	178	70	Extra-Large
25	Montana	6	109	53	Medium
26	Nebraska	4	102	62	Large
27	Nevada	12	252	81	Extra-Large
28	New Hampshire	2	57	56	Medium
29	New Jersey	7	159	89	Extra-Large
30	New Mexico	11	285	70	Extra-Large
31	New York	11	254	60	Large
32	North Carolina	13	337	45	Small
33	North Dakota	3	45	44	Small
34	Ohio	7	120	75	Extra-Large
35	Oklahoma	6	151	68	Large

	Unnamed: 0	Murder	Assault	UrbanPop	UrbanPop_Category
36	Oregon	4	159	67	Large
37	Pennsylvania	6	106	72	Extra-Large
38	Rhode Island	3	174	87	Extra-Large
39	South Carolina	14	879	48	Small
40	South Dakota	3	86	45	Small
41	Tennessee	13	188	59	Medium
42	Texas	12	201	80	Extra-Large
43	Utah	3	120	80	Extra-Large
44	Vermont	2	48	32	Small
45	Virginia	8	156	63	Large
46	Washington	4	145	73	Extra-Large
47	West Virginia	5	81	39	Small
48	Wisconsin	2	53	66	Large
49	Wyoming	6	161	60	Large

Step 7: Save the cleaned file¶

A new csv file has been created with our Data Pre Processing with Urban Population categories and cut offs.

1. Data Cleaning a. Smooth noisy data b. Handling missing data c. Data Wrangling
2. Data Integration
3. Data Transformation
4. Data Reduction
5. Data Discretization (we changed the UrbanPop to categories: small, medium, large, and extra-large)

```
In [28]: # without the index column.
# Hint: to_csv has an index= option.
# Save the cleaned DataFrame to a CSV file
df.to_csv('UrbanPop_Categories.csv', index=False)
```

Cleaning Log

The lab textbook problem 2.1 was completed in 6 - 7 steps.

In step 1. the dataset was loaded.

In step 2. Observations are recorded at first glance.

In step 3. Count the missing value

In step 4. Impute the missing value (median replaced NaN on "Assault")

In step 5. Remove duplicate rows

In step 6. Change noisy data types. (change float to int)

New York changed from 6 to 60.0 %

Iowa changed from 570 to 57 %

North Dakota is a negative outlier at 0 and probably skews the average, 3 is better on "Murder".

We established a relation between urban population category and crime type with a function `UrbanPop_condition()` containing an if-elif-else conditional.

End Of Lab - Behzad Ghabaei

In []: